



slington college
(इस्लिङ्टन कलेज)

CC5068NI Cloud Computing and the Internet of Things

Smart Irrigation and Water Tank Monitoring System

50% Group Coursework

2025 -26 Autumn Semester

Student Name	London Met ID
Aashutosh Subedi	24046336
Ankit Gautam	24046351
Bibek Bhatta	24046541
Raja Khatiwada	24046450
Samriddha Sapkota	24046475

Assignment Due Date: 18/05/2026

Assignment Submission Date: 18/05/2026

I confirm that I understand my coursework needs to be submitted online via MST under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

7% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

Small Matches (less than 8 words)

Match Groups

- 24 Not Cited or Quoted 7%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 1 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 1% **Internet sources**
- 0% **Publications**
- 6% **Submitted works (Student Papers)**

Final Report Submission for Group Coursework

Islington College,Nepal

Document Details

Submission ID
trnoid::3618139431612

Submission Date
May 18, 2026, 4:11 PM GMT+5:45

Download Date
May 18, 2026, 4:24 PM GMT+5:45

File Name
Introduction.docx

File Size
3.9 MB

28 Pages
3,472 Words
19,009 Characters

Acknowledgement

Without help from key people and organizations, finishing this work might never have happened. Gratitude goes to the teaching team at Islington College - steady direction made a real difference. Insight from lecturers at London Metropolitan University shaped progress more than once. Encouragement, offered at tough moments, kept momentum alive. Feedback, always clear, helped refine each stage along the way.

Gratitude extends to the open-source world - tools like the Arduino IDE, ThingSpeak, Fritzing, Circuit Designer, and GitHub enabled what once seemed technically out of reach. When hurdles emerged during setup, support came not from manuals alone, but from shared knowledge online. Discussion threads, scattered guides, and public code repositories eased the path forward. Especially helpful: crowd-sourced insights on the ESP32 DevKit V1 and connected parts. Without such collective effort, progress would have slowed sharply.

Grateful credit belongs to the team behind ThingSpeak's IoT system - offering free cloud access - that opened doors for learners working under tight financial limits. Their support shaped what this effort could become.

Gratitude goes to our classmates and teammates, whose steady involvement shaped every phase of this work. Because of consistent input from everyone, tasks like planning, building, and recording progress moved forward without delay. Completion came not by chance - but through repeated coordination across all parts. Shared commitment turned separate efforts into a finished outcome.

Abstract

Small farmers in Nepal struggle with water management due to their resource-intensive and infeasible manual irrigation methods. With that in mind, this project offers a Smart Irrigation and Water Tank Monitoring System built around the ESP32 DevKit V1 (an affordable but powerful microcontroller) to greatly simplify the process. The system uses irrigation automation to decrease the available manpower and control irrigation while providing updates on the water level of the tank.

Automated irrigation is triggered in case moisture level of the soil is at or below 35% and the available water in the tank is above 20%. In this case, the water tank is emptied, and the 12V submersible pump is activated. The moisture tank is monitored by the high-level water of the tank and with the help of photoelectric sensors, which control the level of water. The irrigation system consists of a floating taut-line numerous receiver (RGOut) level control valve. The moisture level is monitored by an ultrasonic level sensor.

The system is configured with the help of ThingSpeak online service. The moisture tank is monitored by the ThingSpeak and the cloud house irrigation system is monitored. The system is based on the Agile iterative methodology allowing advancements in the area's three main sections: cloud-based irrigation, sensors, and control. The results are based on iterative sensor and irrigation testing and are reflective of the system's cloud irrigation and sensor control. The cost and scale of the solution fit the agricultural climate and the goal is to implement smart farming at the state's level.

Table of Contents

1. Introduction.....	1
1.1. Project Overview.....	1
1.2. Current Scenario.....	2
1.3. Problem Statement.....	3
1.4. Aim and Objectives	4
2. Background and System Design	5
2.1. System Overview.....	5
2.2. System Design and Architectural Diagrams.....	6
2.3. Block Diagram.....	6
2.4. Detailed Requirement Analysis	7
2.4.1. Hardware Components	7
2.4.2. Software and Tools	8
2.5. Individual Contribution plan.....	9
3. Development	10
3.1. Flowchart of the system.....	11
3.2. Circuit diagram	12
3.3. System Schematic.....	13
3.4. System Development.....	14
3.4.1. Pin Configuration.....	19
4. Results and Findings	20
4.1. Test Environment.....	20
4.2. Test Cases.....	20
4.2.1. Test 01: Capacitive soil sensor test.	20
4.2.2. Test 02: Ultrasonic Sensor test.	22
4.2.3. Test 03: Relay module test.	24
4.2.4. Test 04: ThingSpeak Test.....	25
4.2.5. Test 05: To test Blynk Server Monitoring	26
4.3. Key Findings.....	27
5. Further Work	28
6. Conclusion.....	30
7. References	31

8. Appendix.....	32
8.1. Appendix A: Source code for the ESP 32.....	32

List of Figures

Figure 1: System Design and Architectural Diagram	6
Figure 2: Block Diagram.....	6
Figure 3: Flowchart of the system.	11
Figure 4: Circuit Diagram of the system	12
Figure 5: System Schematic.	13
Figure 6: Connecting Soil Moisture sensor to ESP32	14
Figure 7 Adding Ultrasonic Sensor	15
Figure 8 Adding Relay Module	16
Figure 9 Adding water pump	17
Figure 10: Blynk Dashboard	18
Figure 11: Thingspeak dashboard	18
Figure 12: Soil Moisture Test	21
Figure 13: Ultrasonic sensor test.	23
Figure 14: Relay Module test.	24
Figure 15: ThingSpeak Dashboard	25
Figure 16: Blynk Dashboard.	26

List of Tables

Table 1: Hardware Components	7
Table 2: Tools and Software used.....	8
Table 3: Table of Pin Components	19
Table 4: Test 1	20
Table 5: Test 2	22
Table 6: Test 3	24
Table 7: ThingSpeak Test.....	25
Table 8: Blynk Server Monitoring.	26

1. Introduction

1.1. Project Overview

This project introduces a working model of an automated watering and reservoir tracking setup based on the ESP32 DevKit V1. Instead of manual checks, moisture readings come from a capacitive probe while tank fill height is judged using sound wave reflection measured by an HC-SR04 unit. Power flows to a 12-volt underwater pumping device only when conditions demand, thanks to control signals sent through a five-volt relay circuit. Information skips cables entirely with WiFi embedded within the board which pushes updates straight to ThingSpeak online. Once there, values get saved, turned into graphs, then opened up for viewing anytime from phones or browsers far away.

Beginning with detection, the soil moisture sensor captures real-time water levels within the ground at regular intervals. Once values fall beneath a set limit, the ESP32 evaluates reservoir status using data from an ultrasonic device. Should liquid volume meet minimum requirements, the relay engages, powering the pump for a predetermined duration. In contrast, if storage runs too low, activation halts while a warning transmits directly to the online platform. Rather than pause updates during such cases, measurements continue streaming to ThingSpeak every quarter-minute interval. Through this flow, cultivators gain constant visibility into earth dampness alongside supply availability, regardless of physical proximity.

This solution tackles inefficient hand-operated watering alongside unseen storage tank issues, both within one working model. Though separate, the challenges connect through resource oversight. A single design now handles both by removing human timing errors while adding visibility into reserve levels. Instead of treating them apart, the approach unites control and awareness where waste often occurs. Through integrated feedback, adjustments happen without waiting for shortages.

1.2. Current Scenario

In Nepal, farming holds up much of the economic activity. Backed by data from the World Bank in 2023, nearly three-tenths of national output comes from this sector, while close to two out of every three workers earn livelihoods here in Nepal - most being small-scale growers relying on rain-fed plots across flat southern stretches and rolling hills (World bank Group, 2023). Even so, systems for watering crops stick mostly to older methods. As noted by the UN's FAO, spreading liquid via uncontrolled flooding or shallow ditches continues to be standard practice on most land under cultivation; because of poor control, losses from seepage, air exposure, and excess delivery might drain away anywhere between four and six out of ten drops drawn for field use (FAO, 2021).

Meanwhile, supplies of fresh water are harder to anticipate. Because monsoon cycles in Nepal shift more wildly due to climate shifts, rains arrive without pattern. This uncertainty threatens farming methods tied strictly to wet seasons. Instead of depending only on downpours, drawing irrigation from lakes, tanks, or ponds has become more common. Yet most growers lack even basic tools to track what volume stays within these reserves.

Most of the world's fresh water goes to farming i.e. about seventy percent, according to UNESCO data from 2021 (UNESCO, 2021). Irrigation upgrades can slash that usage by up to half, studies such as one by the USDA in 2020 show (USDA, 2026). Devices linked through digital networks now guide watering decisions on major farms across nations including Israel and the Netherlands. Fields there rely on ground probes, climate monitors, targeted drips, and remote computing platforms. Yet many smaller producers remain far behind; take Nepal, where affordability, ease of setup, and weak internet access shape what tools work.

This work tackles the problem head-on. Using inexpensive parts found almost everywhere - like the ESP32 chip costing between four and six dollars, the HC-SR04 at one to two bucks, and soil sensors running one to three - it takes shape. Add a no-fee online system, ThingSpeak, and suddenly there is something real. Not just theory,

but a working example of smart watering that fits how farming works in Nepal. Hence, it functions outside labs and can grow when needed.

1.3. Problem Statement

What drives us building this system comes down to these issues:

- Watering crops by hand often lacks precision in Nepal. Instead of using sensors, farmers usually follow fixed schedules or guess based on appearance. Because of this, fields sometimes get too much water saturating roots, encouraging rot, and squandering supply. At other times, plants receive too little, leading to strain, lower harvests, even total loss when conditions worsen. Without real-time measurements guiding decisions, adjustments remain reactive rather than informed.
- Most farmers using stored water cannot see current levels inside tanks or reservoirs. Without clear data, they often check by hand, a slow method prone to error. As a result, many start irrigation too late, either because the supply runs low or drops unexpectedly. When motors keep running with little water left, overheating occurs, leading to burned-out systems. A monitoring device that updates continuously removes uncertainty from daily decisions. Problems like under-watering crops or damaged equipment become avoidable.
- Working the land takes effort especially when water must be managed by hand each day. On tiny farms spread across scattered patches, walking to every site eats up hours. Without machines to help, someone has to be there in person so when these problems arise the project comes into play; where decisions happen only on site and in real time.
- When past records are missing, spotting trends in soil wetness or how much water gets used becomes a guesswork. Because nothing tracks what happens week after week, fine-tuning everything would be near impossible. Smart irrigation generates the data necessary to eventually reduce water usage.

Put simply, today's irrigation practices respond too late, rely on hands-on control, lose resources needlessly, while staying out of sight. Instead, this project acts ahead, runs without constant human input, tracks usage precisely and operates under remote observation.

1.4. Aim and Objectives

One goal drives this project: building an IoT system with two sensors powered by an ESP32 chip. Real-time readings from soil guide automatic watering decisions. Instead of manual checks, moisture levels trigger responses without delay. Meanwhile, an ultrasonic device tracks how much water remains in storage tanks. This detail matters just as much as the ground conditions. Data flows continuously into the ThingSpeak online service. Remote access allows users to watch changes over time. Alerts appear when thresholds are crossed. Automation replaces guesswork throughout the process. The setup links physical inputs with digital feedback seamlessly.

2. Background and System Design

2.1. System Overview

The system operates as a closed-loop IoT application consisting of three functional domains: data collection at the field, cloud-based processing and storage and user-facing monitoring.

Down at the field layer, one ESP32 DevKit V1 handles both device control and wireless communication. Every fifteen seconds, it checks data from a capacitive soil probe linked to its analog input. Instead of using "AND" signals move through separate steps - first, a short burst fires into the HC-SR04's trigger; next, the return wave gets measured. From that interval, distance emerges in cm, later transformed into a tank fill percentage. When earth moisture dips under forty percent while liquid reserve stays past twenty percent, activation happens after validation. A high signal appears on a designated output, switching on a mechanical relay via electrical contact. That switch turns power on to a twelve-volt underwater pump. Once duration ends, voltage drops back down, breaking circuit flow and halting delivery. In cases where reservoir volume slips too low, the operation gets blocked automatically then the actuator ignores start requests, simultaneously marking status online inside the monitoring feed.

Every 15 seconds, sensor measurements like the soil moisture, water tank level and pump operation are sent through MQTT to a ThingSpeak channel. Stored by ThingSpeak as timestamped entries, the information appears instantly on web-based graphs viewable from any device. When readings exceed preset limits, automated notifications trigger via email or HTTP callbacks, using built-in tools like PaniHTTP and React. These responses occur without manual input once rules are set.

Power comes into the whole setup through a 12V DC adapter. From that source, a buck converter or possibly a USB power bank steps it down to 5V for the ESP32's built-in regulator. That microcontroller draws its energy from the converted USB-type feed. Meanwhile, the relay module operates on its own 5V supply. It handles the 12V pump by interrupting the positive lead going to the motor. Only when triggered does current flow to drive the pump.

2.2. System Design and Architectural Diagrams

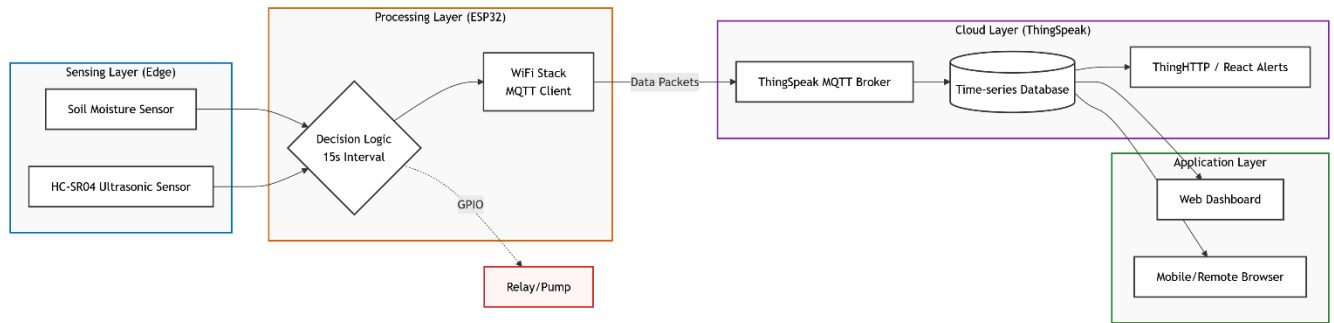


Figure 1: System Design and Architectural Diagram

2.3. Block Diagram

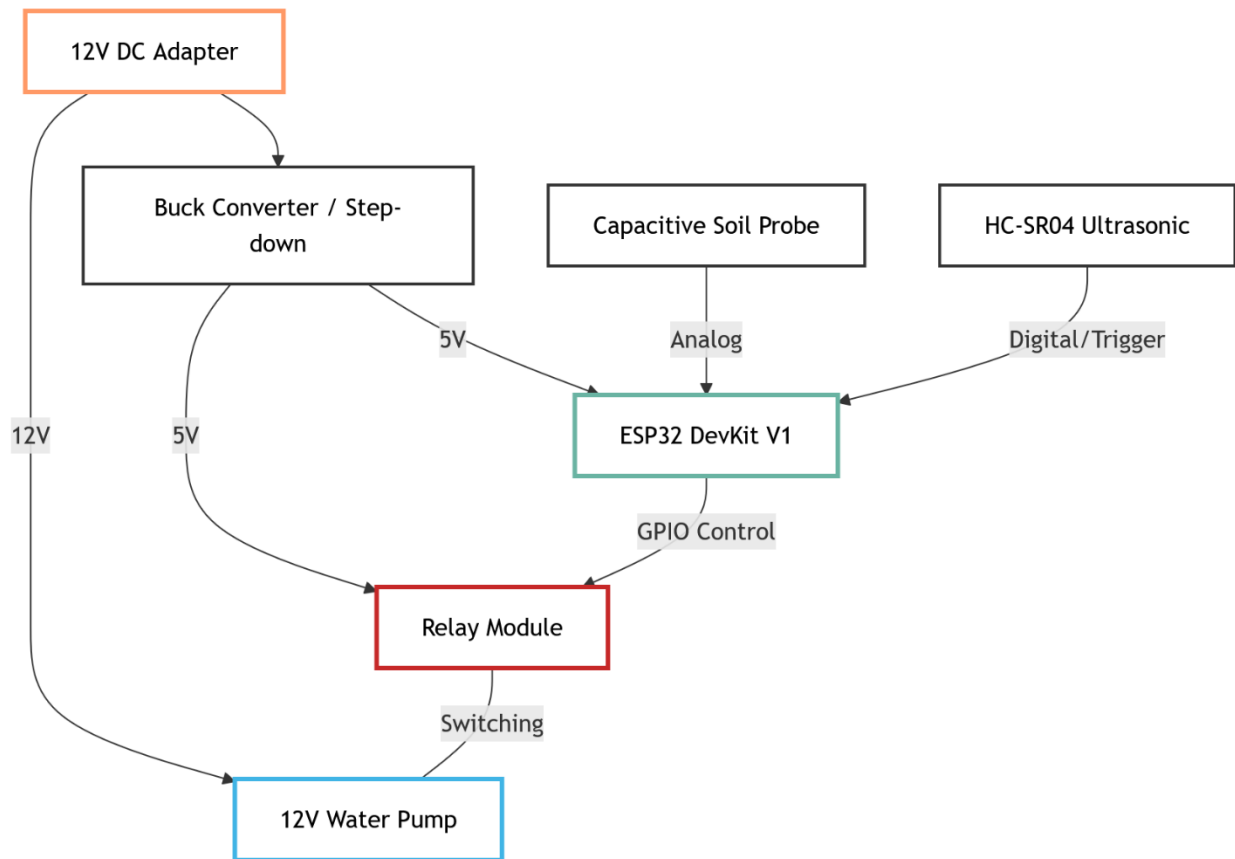


Figure 2: Block Diagram

2.4. Detailed Requirement Analysis

2.4.1. Hardware Components

The following hardware components are used, along with their functions in the system:

Component	Purpose
ESP32 DevKit V1	Central microcontroller; Wi-Fi and MQTT
Capacitive Soil Moisture Sensor	Real-time soil moisture measurement
HC-SR04 Ultrasonic Sensor	Water tank level detection via echo
5V Single-Channel Relay Module	Controls 12V pump via GPIO signal
Submersible Mini Water Pump (12V)	Delivers water to soil when triggered
12V DC Adapter	Primary power source
Breadboard and PCB	Prototyping and circuit assembly
Jumper Wires (M-M, M-F, F-F)	Component interconnections

Table 1: Hardware Components

2.4.2. Software and Tools

The following software and tools are used in this project:

Tool / Platform	Purpose
Arduino IDE	Firmware development and serial monitoring for ESP32
ThingSpeak	Cloud IoT platform for data storage, visualisation, and alerts
MQTT Protocol	Lightweight publish-subscribe messaging for sensor data transmission
Blynk App	Mobile dashboard for remote monitoring and control
Fritzing	Circuit schematic and PCB design diagrams
Circuit Designer	Online simulation and circuit testing before physical build
GitHub	Version control and collaborative code management
draw.io	Architectural diagram and flowchart creation

Table 2: Tools and Software used.

2.5. Individual Contribution plan

This project was executed as a group of five members, with responsibilities distributed equitably across the core domains of hardware, software, cloud integration, testing, and documentation. The following table outlines the contribution plan assigned to each member:

Member Name	Primary Responsibilities
Aashutosh Subedi	Hardware assembly, circuit design and Fritzing schematic, component sourcing and breadboard prototype
Ankit Gautam	ESP32 firmware development in Arduino IDE, sensor calibration, relay and pump control logic
Bibek Bhatta	ThingSpeak cloud integration, MQTT configuration, channel setup and automated alert configuration
Raja Khatiwada	System testing, test case design, results analysis, Circuit Designer simulation and debugging
Samriddha Sapkota	Report writing, architectural diagrams, flowchart development, GitHub repository management and documentation

3. Development

The methodology for this project is: Agile iterative methodology.

Working in a group of five makes this approach useful, since tasks can unfold at once while staying connected. Starting with separate chunks like sensing, linking, and moving, the team builds code on GitHub, sets up data paths through ThingSpeak, and puts together hardware relays all within the same window of time. Progress moves faster when one part doesn't block another; for instance, checking if the HC-SR04 reads water depth right happens long before pump decisions are made. Testing early pieces independently keeps delays low across the whole effort.

Starting with small adjustments, repeated testing helps fine-tune how the system responds when linked to actual equipment. Because performance depends on fixed physical points: like soil moisture at 40% or water levels dropping to 20%, going through multiple rounds of trial lets the group check responses under realistic conditions prior to advancing. Through this method, decisions built into the software become more stable while safeguarding components against damage such as overheated pumps or corroded sensors. As a result, what emerges is a working model better aligned with both research goals and practical demands.

3.1. Flowchart of the system

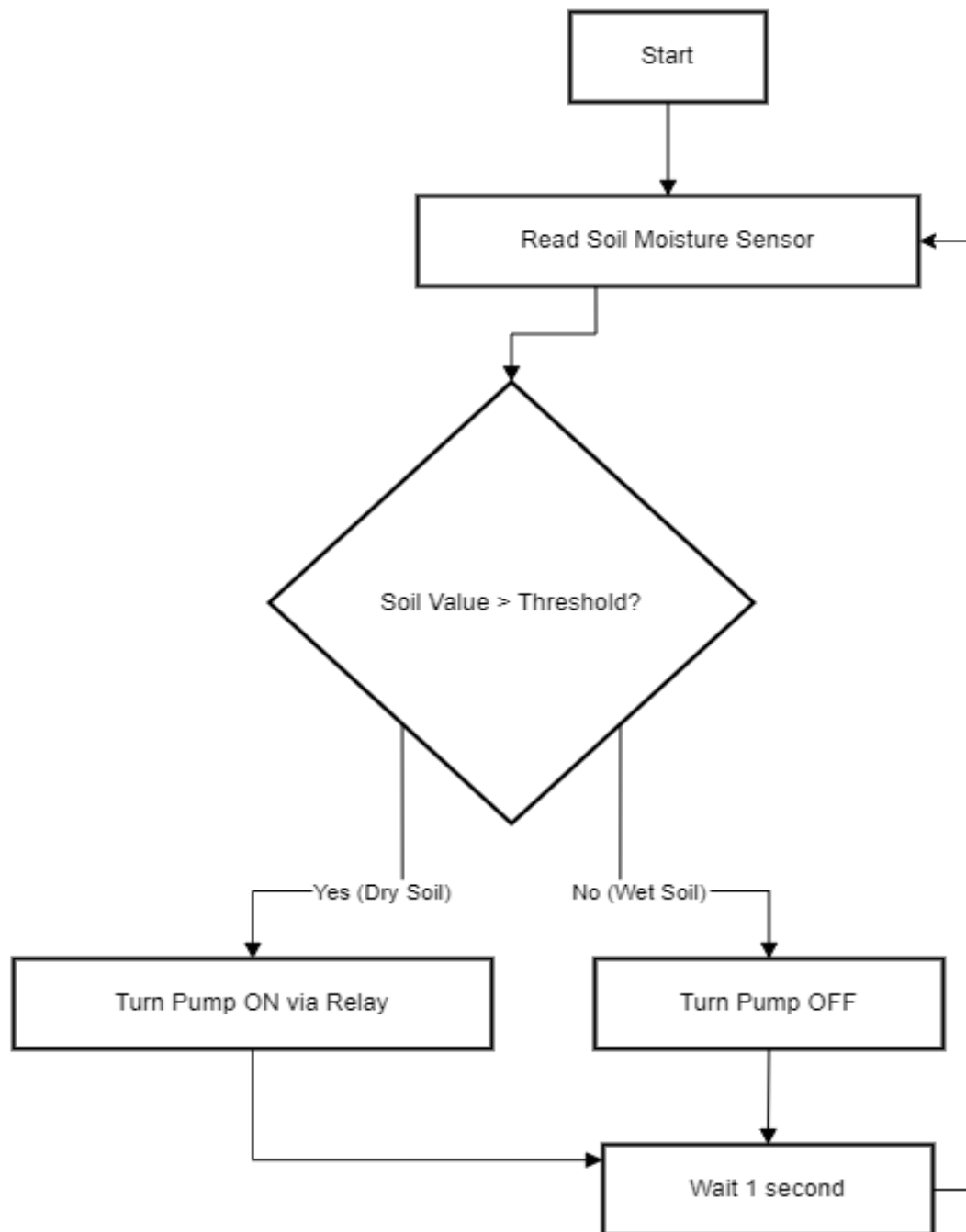


Figure 3: Flowchart of the system.

3.2. Circuit diagram

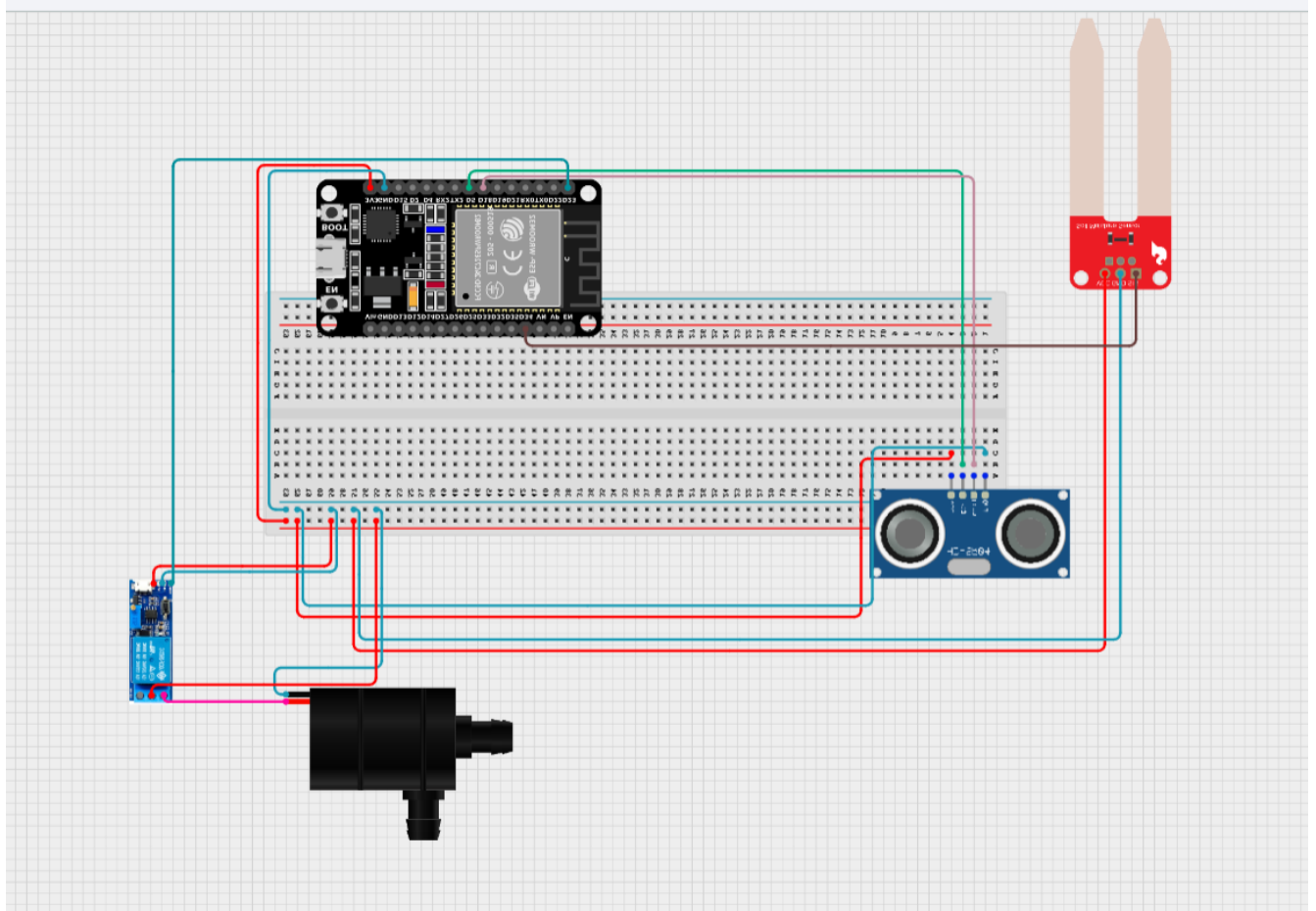


Figure 4: Circuit Diagram of the system

3.3. System Schematic

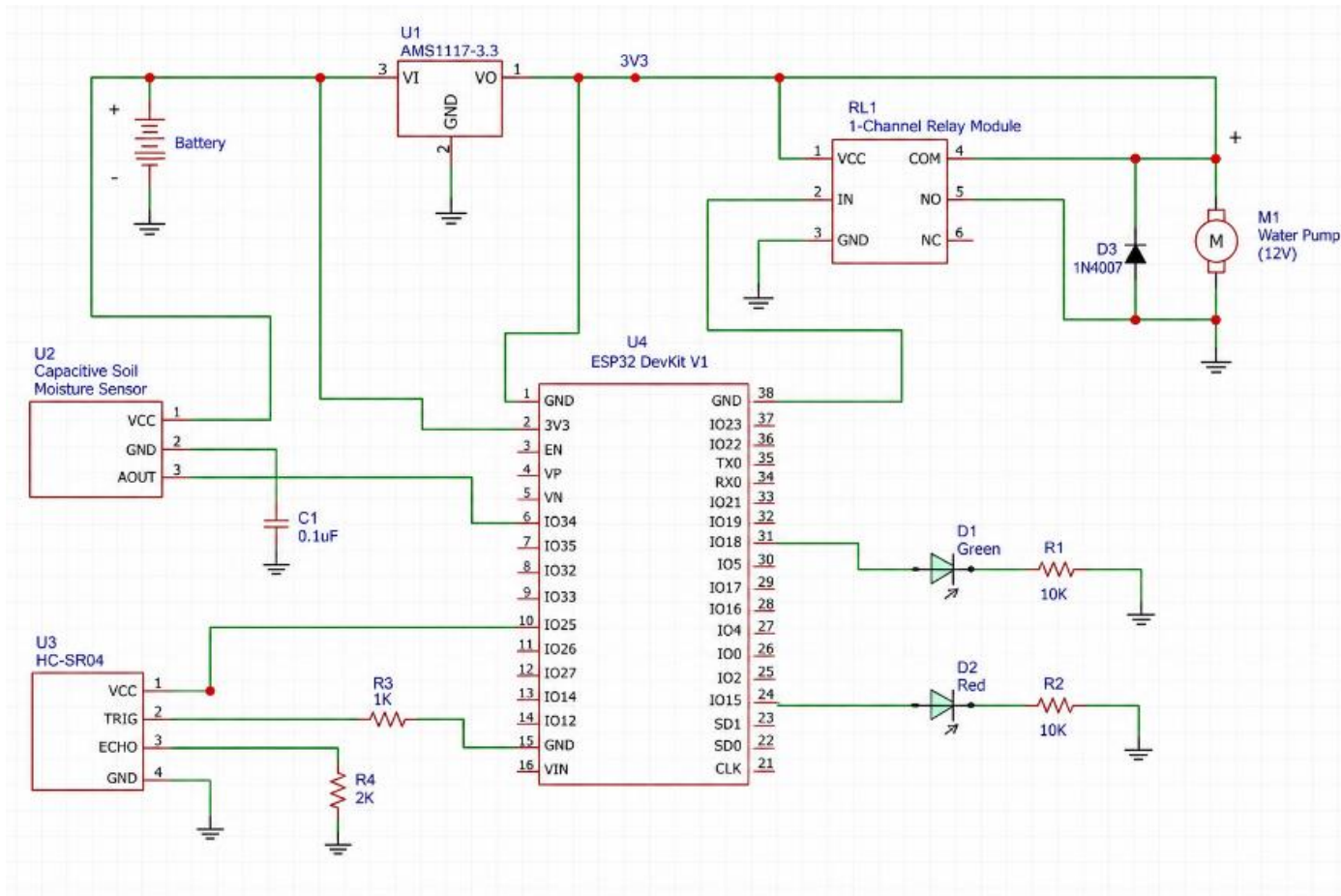


Figure 5: System Schematic.

3.4. System Development

Step 1: Connecting ESP32 to soil moisture sensor.

The ESP32 is securely positioned onto the breadboard, with the 3.3V/Vin and GND of ESP32 connecting to the positive and negative power rails of the breadboard running parallelly at the top and bottom respectively. Then, the VCC and GND of the soil moisture sensor are wired into the positive and negative power rails of the breadboard respectively. The signal/output pin of the soil moisture sensor is wired into an analog pin of ESP32. Power supply is provided from the laptop through the USB connection.

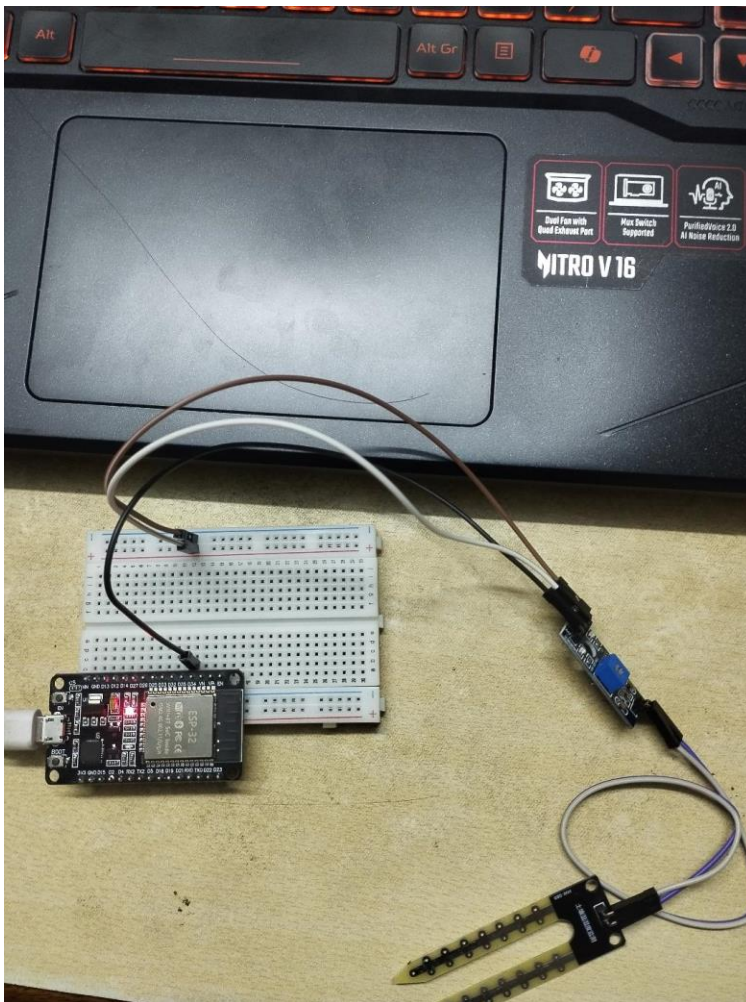


Figure 6: Connecting Soil Moisture sensor to ESP32

Step 2: Connecting ultrasonic sensor to ESP32.

The ultrasonic sensor is attached to the breadboard for reliable installation. Its VCC terminal is connected to the 3V3 of ESP32 and the GND terminal is connected to the (-ve) rail of the breadboard. The TRIG terminal is connected to one digital output terminal of the ESP32 board (for instance GPIO5), while the ECHO terminal is connected to one digital input terminal of the ESP32 (for instance GPIO18). Such connection enables the ESP32 board to send trigger pulse and receive echo pulse, and then calculate distance, which will be used to track the water level inside the storage tank.

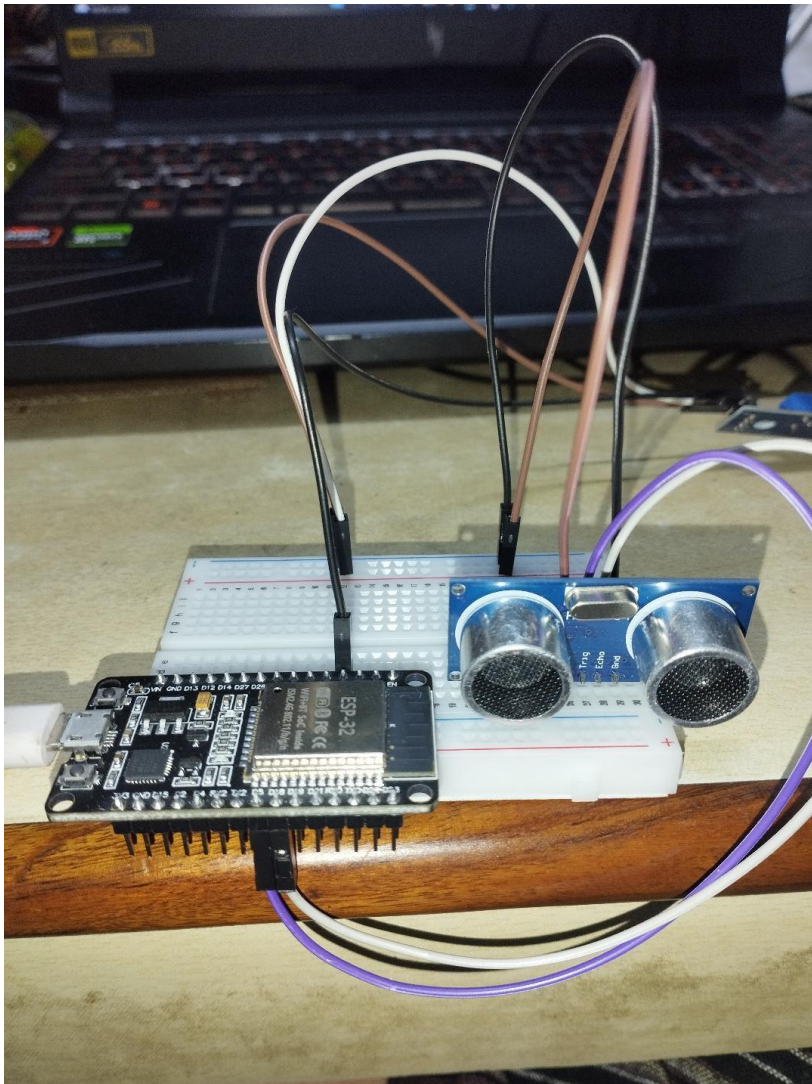


Figure 7 Adding Ultrasonic Sensor

Step 3: Connecting Relay Module to ESP32.

The relay module is placed for controlling the water pump through electrical switching. The VCC terminal of the relay module is connected to the 3V3 of ESP 32, while the GND terminal of the relay module is connected to the (–ve) power rail of the breadboard.

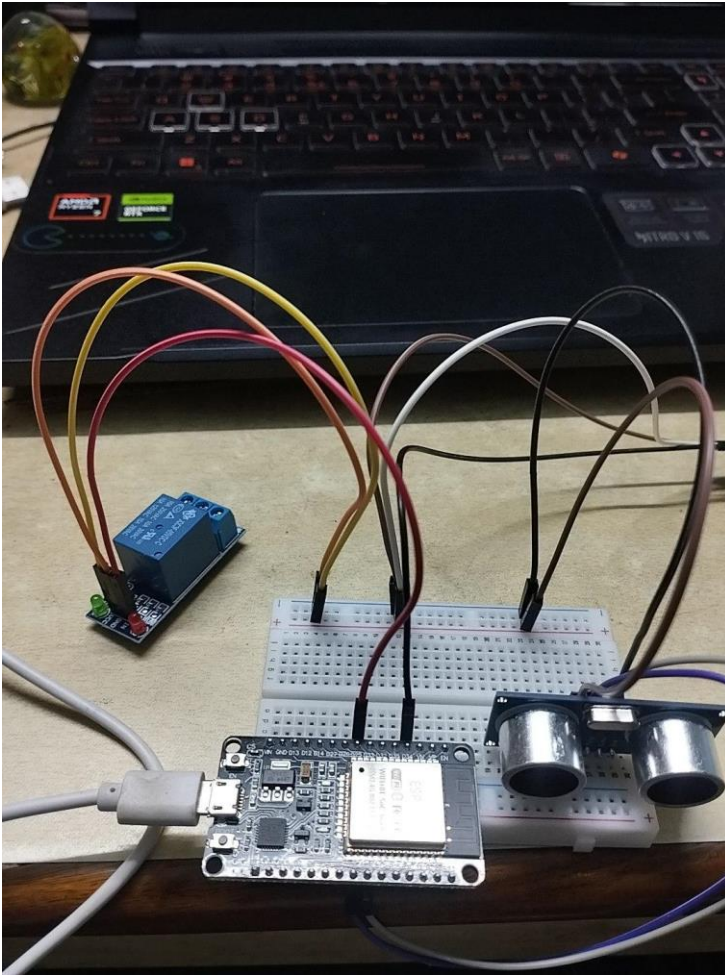


Figure 8 Adding Relay Module

Step 4: Connecting water pump.

The water pump is externally powered through the relay module. One end of the water pump is attached to the Normally Open (NO) contact of the relay, while the other end is connected to external power supply.. This will ensure that the water pump gets power only if the relay is activated by the ESP32.

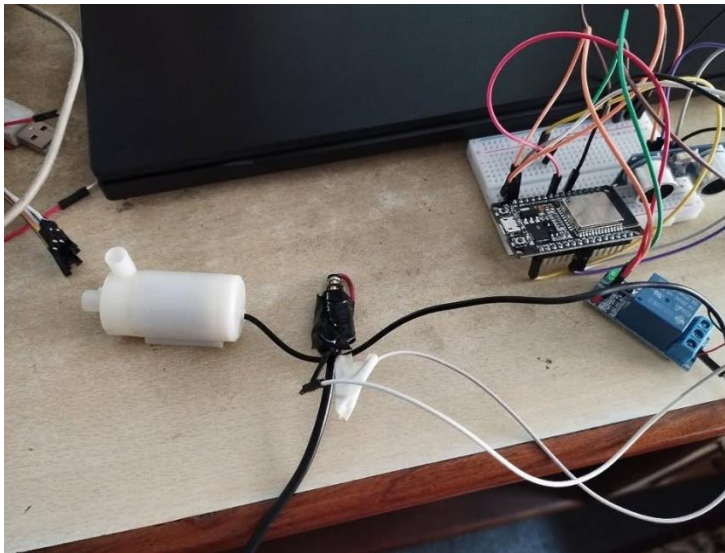


Figure 9 Adding water pump

Step 5: connecting to cloud

After connecting all the devices the data is send to Blynk and thingspeak. In which thingspeak stores the readings and Blynk Displays the current status of the devices.

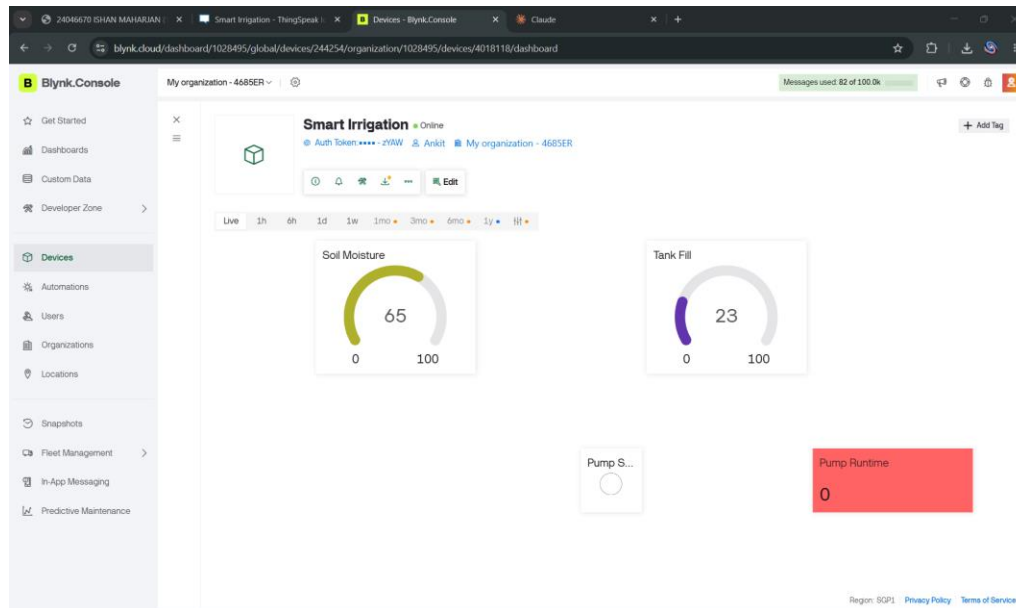


Figure 10: Blynk Dashboard

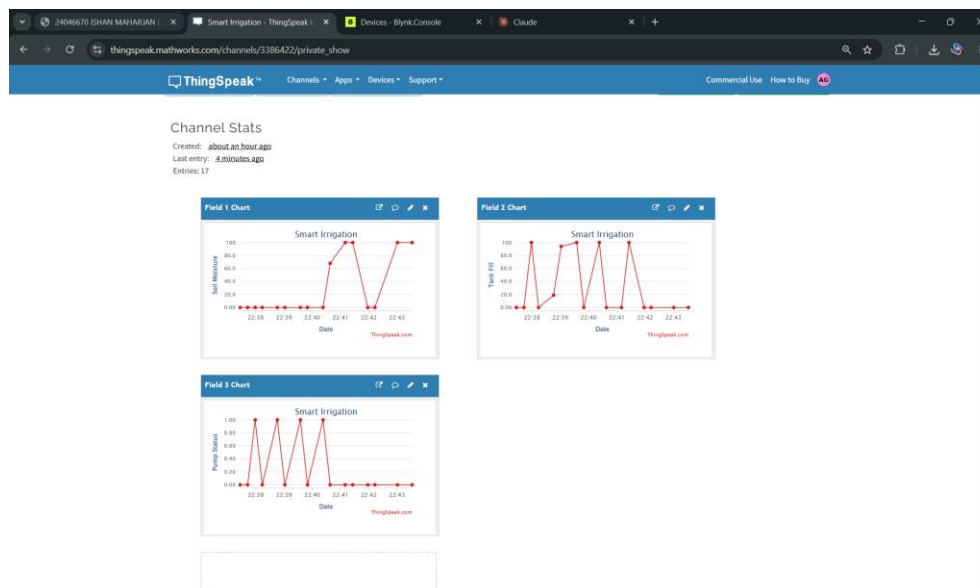


Figure 11: Thingspeak dashboard.

3.4.1. Pin Configuration

Component	Pin Name	Connected To
Soil Moisture Sensor	VCC	VN ESP32
	GND	Breadboard (–ve) rail
	Signal / A0	ESP32
Ultrasonic Sensor (HC-SR04)	VCC	VN ESP32
	GND	Breadboard (–ve) rail
	TRIG	ESP32(D5)
	ECHO	ESP32(D18)
Relay Module	VCC	3V3 ESP32
	GND	Breadboard (–ve) rail
	IN	ESP32(D25)
Water Pump	Positive Terminal	Relay NO terminal
	Negative Terminal	External supply negative
Relay Switching Terminals	COM	External power positive
	NO	Pump positive terminal
ESP32	3.3V / Vin	Breadboard
	GND	Breadboard (–ve) rail

Table 3: Table of Pin Components

4. Results and Findings

4.1. Test Environment

A plastic bin holding damp earth sat on a lab table, mimicking real-world terrain through measured saturation levels. Above it, a clear reservoir equipped with an ultrasonic module hovered in fixed position. This HC-SR04 unit sent signals downward while linked to a microcontroller below. The device ran on stable power, tied into household wireless internet at standard frequency bands. Information streamed continuously to a cloud-based tracking platform during trials. Readings also appeared line by line on a computer screen beside the setup. Engineers watched each value scroll past in real time using basic monitoring software.

4.2. Test Cases

4.2.1. Test 01: Capacitive soil sensor test.

Test	1
Objective	To check the capacitive soil moisture sensor correctly reads soil moisture levels and transmits values to the ESP32.
Activity	The soil moisture sensor's GPIO 34 (ADC input) was connected to the ESP32. Both water-saturated soil and completely dry soil were tested. ESP32 was lined with the Arduino IDE Serial Monitor and was powered by USB.
Expected Result	Soil moisture levels will appear as continuous values from 0 to 100 percent in the Serial Monitor. The range for dry readings is from about 70% to 100%, while saturated readings will range from 0% to 20%.

Table 4: Test 1

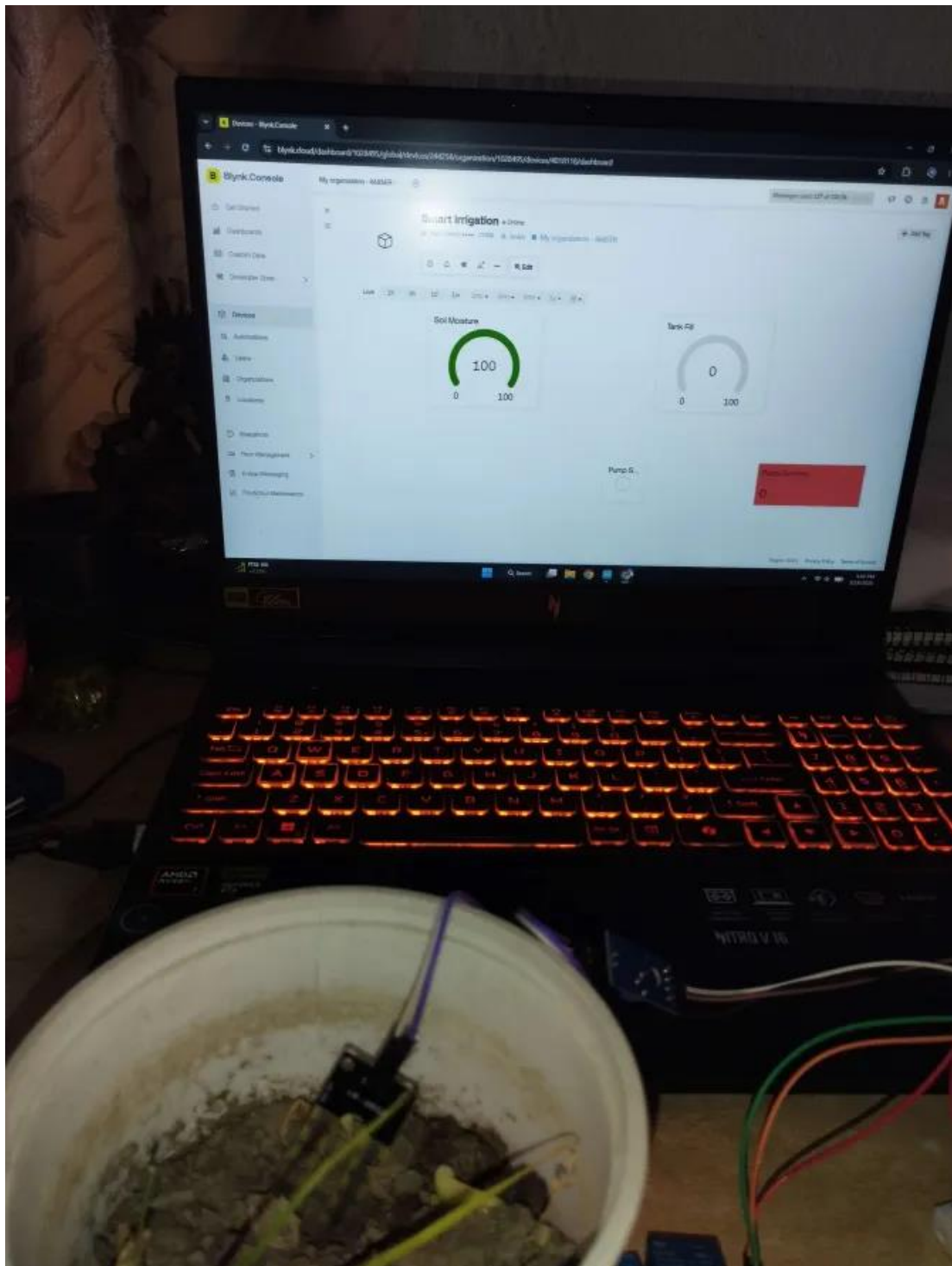


Figure 12: Soil Moisture Test

4.2.2. Test 02: Ultrasonic Sensor test.

Test	2
Objective	To check the HC-SR04 ultrasonic sensor accurately measures water tank distance and converts it to a fill percentage.
Activity	The trigger pin of the HC-SR04 was connected to GPIO 5 and the echo pin to GPIO 18 (with a resistor divider). A test container was mounted to the sensor. Three known water levels (empty, half, full) were marked with a ruler.
Expected Result	For water level readings (distance) in the Serial Monitor, the allowed error will be about 1 cm. Consequently, the expected Serial Monitor display will be as follows: tank water level empty = 0%, tank water level half = 50%, tank water level full = 100%.

Table 5: Test 2

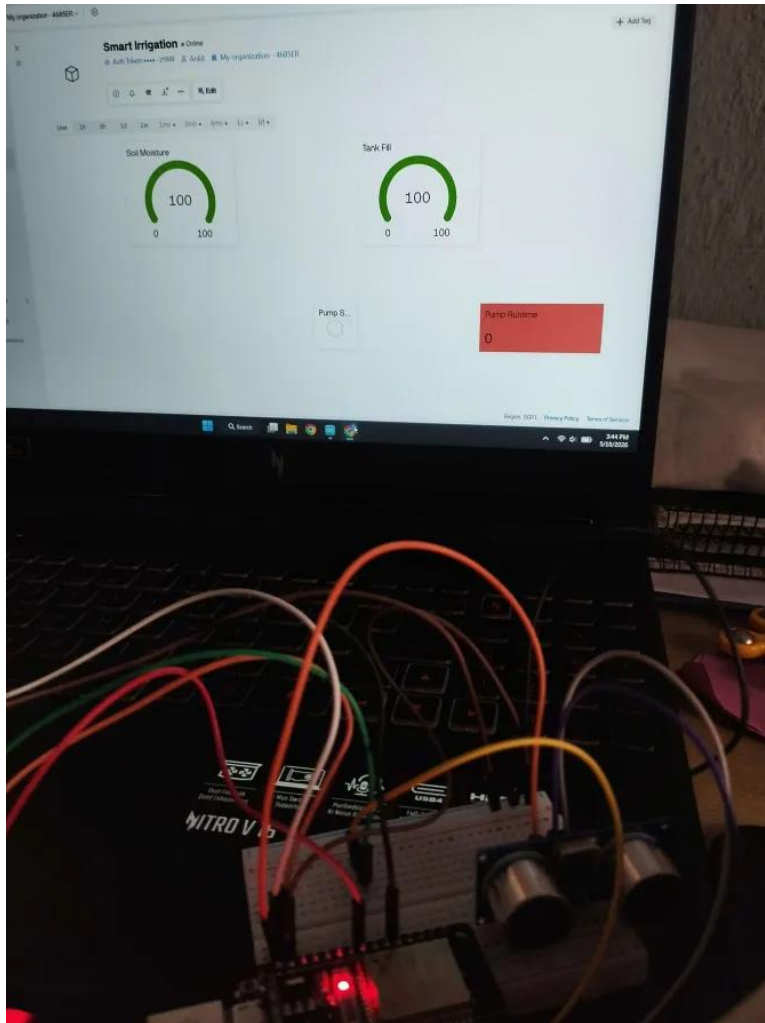


Figure 13: Ultrasonic sensor test.

4.2.3. Test 03: Relay module test.

Test	3
Objective	To test the relay module activates the water pump when soil moisture drops below the 40% threshold and tank level is above 20%.
Activity	Soil moisture was set to 32% by partially drying the test soil. The tank level was kept at 75 percent. The ESP32 was allowed to run the decision-making logic for one whole polling cycle (15 seconds).
Expected Result	After the relay is switched ON (an audible click will be heard), the relay will pump water for 30 seconds, during which GPIO 26 will be HIGH. Pumping (Field 3) will be set to 1 in ThingSpeak.

Table 6: Test 3

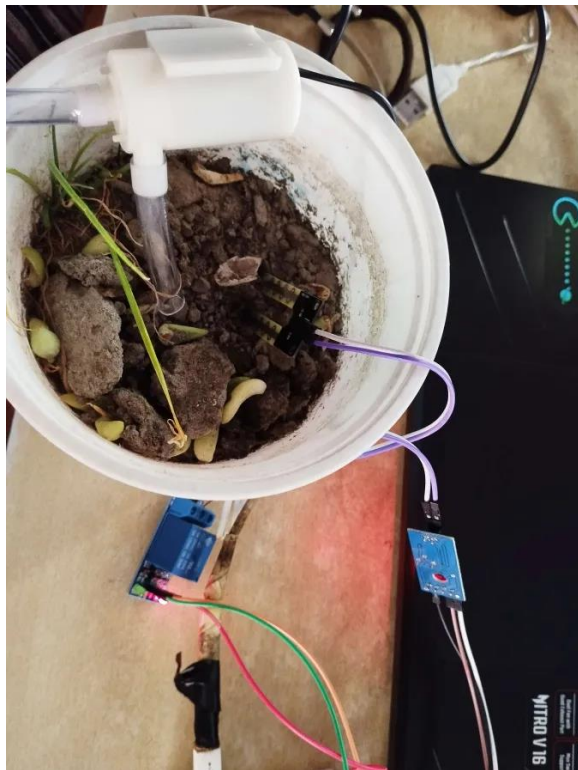


Figure 14: Relay Module test.

4.2.4. Test 04: ThingSpeak Test

Field	Description
Test	4
Objective	To test whether the ESP32 correctly uploads soil moisture, tank level, and pump status data to the ThingSpeak cloud platform every 15 seconds.
Activity	The system was connected to a Wi-Fi network and allowed to operate continuously for several polling cycles. Soil moisture and tank levels were manually varied while monitoring updates on the ThingSpeak dashboard and Serial Monitor.
Expected Result	Sensor readings and pump status values will successfully appear on the ThingSpeak dashboard at approximately 15-second intervals. Field 1 will display soil moisture percentage, Field 2 will display tank fill percentage, and Field 3 will display pump status values of either 0 or 1 depending on pump operation.

Table 7: ThingSpeak Test.

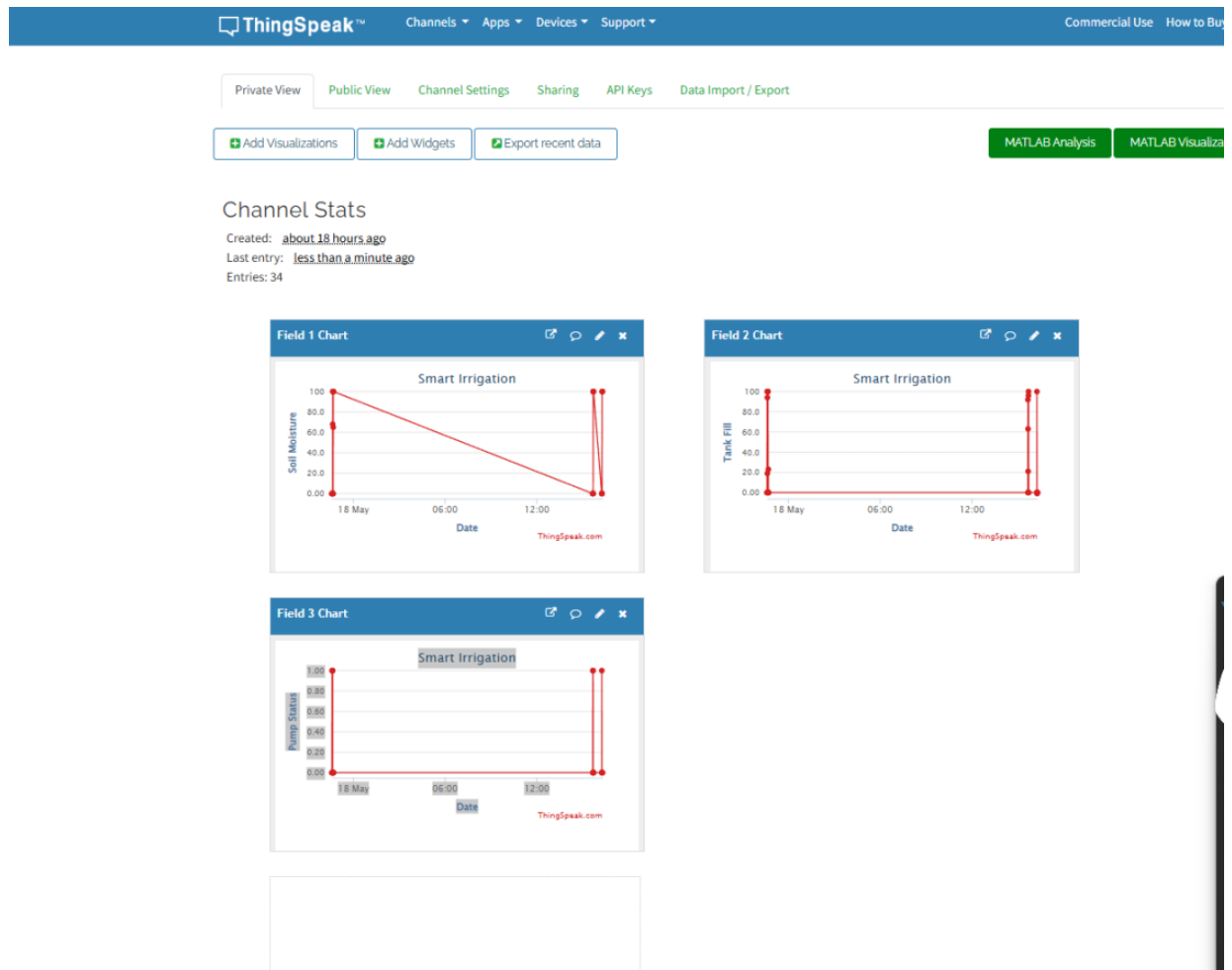


Figure 15: ThingSpeak Dashboard

4.2.5. Test 05: To test Blynk Server Monitoring

Field	Description
Objective	To test Blynk Server Monitoring
Activity	All sensors were connected to ESP-32. Blynk server was connected to ESP-32. DataStream widgets were created for each sensor.
Expected Result	Sensor values should be displayed into different widgets in Blynk.
Actual Result	Sensor values were displayed into different widgets of Blynk.
Conclusion	Test was successful

Table 8: Blynk Server Monitoring.

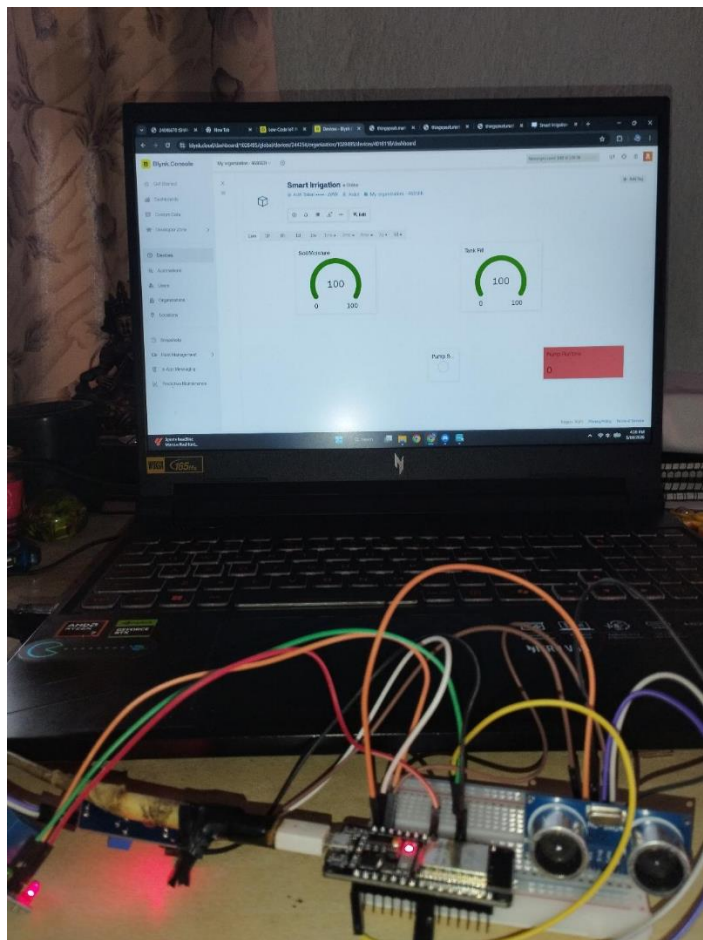


Figure 16: Blynk Dashboard.

4.3. Key Findings

The system of sensors, ESP32, and relay worked together seamlessly. Sensing was accurate, and the system processed information without errors. Signals and responses flowed naturally. Build also shows that the system can perform real-world tasks such as watering plants and managing water consumption. It is capable of operating outside of the lab. Reality tests proved that the system works, and showed potential. The system showed end-to-end usability thanks to the water flow control and a real working tasks. This system of elements makes sense compared to the basic real-world everyday needs. The advantage of the simple system and elements is that it can be used and synergize without being perfect. Simple tasks and scenarios showed the primitive but practical usefulness of the elements of the setup.

5. Further Work

The current system has met its primary goals and exhibits solid, proof-of-concept performance. Several improvements are suggested for increased functionality, robustness, and ease of use in the real world. These improvements include the following.

- **Weather API Integration:** Adding a weather data API (such as OpenWeatherMap's service) would allow the system to delay irrigation if rain is expected and further conserve water and improve the system's operational logic.
- **Multi-Zone Sensor Network:** The working prototype is comprised of one soil sensor and one ultrasonic sensor. By creating a multi-node architecture with several ESP32 devices or a mesh network, the system could also monitor crop zones that are equally distant but also independent of each other and each with different moisture parameters thresholds.
- **Solar Power Integration:** With a solar panel and a battery in place of the current 12V DC AC adapter, the prototype would also be useful for off-the-grid agricultural sites. This would improve the solutions availability in rural Nepal.
- **Predictive Irrigation:** Once sufficient historical sensor values are continuously uploaded to the cloud servers; machine learning algorithms could be implemented to solve the predictive irrigation problem (moving the system from reactive to predictive water management).
- **Improved Mobile Application:** A tailor made mobile app replacing the Blynk dashboard would allow threshold values to be set and notifications to be sent directly.
- **Long Range or Cellular Connectivity:** If Wi-Fi is not available, a Long Range or a GSM cellular module could replace the ESP32 Wi-Fi chip to extend the system's range from Wi-Fi to remote farm areas.

- Water Flow Meter Integration: By adding a flow meter to the pump of the device it can help while tracking the water levels per irrigation. In turn providing insightful data about the device's status.

6. Conclusion

One step beyond old methods appears here, where tech meets field work without extra cost or complexity for Nepali farms. Instead of guessing when to water, sensors now guide timing using live soil data sent through an ESP32 board into online tracking via ThingSpeak. Not only does it turn irrigation on and off by need, but also keeps tanks from running dry. When rain grows unpredictable, these updates offer control: less splash, more growth. Shifts like this let small harvesters respond ahead of drought, not after damage shows. What once relied on routine checks now runs smarter, shaped by signals rather than schedules.

What stands out is how well the Agile approach worked when applied across repeated cycles in a shared tech setting. Because version tracking was used, alongside live data tools and fast model testing, progress stayed on track despite mixed system demands. A group of five handled these challenges without major setbacks. Even though automation drove the core aim 'smart watering' the structure allows growth through new inputs or smarter forecasting layers. One takeaway lies here: basic internet-connected devices combined with remote computation enable small-scale solutions capable of shifting broader farming outcomes nationwide.

7. References

FAO, 2021. *IRRIGATION AND DRAINAGE MANAGEMENT*. [Online]

Available at: <https://www.fao.org/4/a0021e/a0021e06.htm>

[Accessed 15 May 2026].

UNESCO, 2021. *UN World Water Development Report 2021*, Paris: s.n.

USDA, 2026. *Irrigation & Water Use*. [Online]

Available at: <https://www.ers.usda.gov/topics/farm-practices-management/irrigation-water-use>

[Accessed 15 May 2026].

World bank Group, 2023. *Nepal*. [Online]

Available at: <https://www.worldbank.org/ext/en/country/nepal>

[Accessed 15 May 2026].

8. Appendix

8.1. Appendix A: Source code for the ESP 32

```
// =====  
  
// Smart Irrigation System — ESP32  
  
// Team 1 AI4 · Islington College · CC5068NI  
  
// =====  
  
// Pins:  
  
//  Soil Moisture Sensor → GPIO34 [Analog / ADC1_CH6]  
  
//  HC-SR04 Ultrasonic   → TRIG: GPIO5 · ECHO: GPIO18  
  
//  Relay (Water Pump)   → GPIO25  
  
// =====  
  
// Libraries required (install via Arduino Library Manager):  
  
//  - PubSubClient by Nick O'Leary  
  
// =====  
  
#define BLYNK_TEMPLATE_ID  "TMPL6yqRFh_OY"  
  
#define BLYNK_TEMPLATE_NAME "Smart Irrigation"  
  
#define BLYNK_AUTH_TOKEN   "9PhKFbd1bX-6eVCneKhaqMaljpAWzYAW"  
  
  
#include <WiFi.h>  
  
#include <PubSubClient.h>  
  
#include <BlynkSimpleEsp32.h>
```



```
// ----- WiFi Credentials -----

#define WIFI_SSID    "NTFiber_FC60_2.4G"

#define WIFI_PASSWORD "CK3Gsa94"


// ----- ThingSpeak MQTT -----

// Create a free ThingSpeak channel, then grab these values from:

// ThingSpeak → Channel → API Keys / MQTT API

#define MQTT_SERVER    "mqtt3.thingspeak.com"

#define MQTT_PORT      1883

#define CHANNEL_ID     "3386422"    // numeric, e.g. "2345678"

#define MQTT_CLIENT_ID "LD0YEx4bKQ88DTQJKzASljk"    // from ThingSpeak
MQTT creds

#define MQTT_USERNAME  "LD0YEx4bKQ88DTQJKzASljk"    // from ThingSpeak
MQTT creds

#define MQTT_PASSWORD  "lieO03LqL4WNNXKPINAufuEx"    // from ThingSpeak
MQTT creds

// ThingSpeak fields:

// Field 1 — Soil Moisture (%)

// Field 2 — Tank Fill (%)

// Field 3 — Pump Status (1 = ON, 0 = OFF)


// ----- Pin Definitions -----
```

```
// NOTE: Changed from GPIO39 to GPIO34 per coursework Test 01 specification.

#define SOIL_PIN  39  // Soil moisture analog input (ADC1 channel only — safe for
analogRead)

#define TRIG_PIN   5  // HC-SR04 Trigger

#define ECHO_PIN  18  // HC-SR04 Echo

#define RELAY_PIN 25  // Relay IN (active-low: LOW = pump ON)


// ----- Thresholds -----

// NOTE: DRY threshold updated from 30% to 40% per coursework specification

// (System Overview section and Test 03 both specify 40%).

#define SOIL_DRY_THRESHOLD  40  // % — pump turns ON  when moisture
BELOW this

#define SOIL_WET_THRESHOLD  60  // % — pump turns OFF when moisture
ABOVE this


#define TANK_LOW_CM      20  // cm — water is LOW  when distance > this

#define TANK_FULL_CM     5   // cm — water is FULL when distance < this


// ----- Timing -----

// NOTE: Updated from 2000ms to 15000ms per coursework specification

// ("every 15 seconds" stated in System Overview and Section 3.1).

#define READ_INTERVAL    15000 // ms between sensor reads (15 s)
```

```
// NOTE: 30-second pump cap added per Test 03 expected result:

// "relay will pump water for 30 seconds".

#define PUMP_MAX_DURATION 30000 // ms — hard safety cut-off per pump cycle


// ----- State -----

bool      pumpRunning  = false;

unsigned long pumpStartTime = 0;


WiFiClient espClient;

PubSubClient mqttClient(espClient);


// =====

// SETUP

// =====

void setup() {

  Serial.begin(115200);

  delay(500);


  pinMode(TRIG_PIN, OUTPUT);

  pinMode(ECHO_PIN, INPUT);

  pinMode(RELAY_PIN, OUTPUT);


  digitalWrite(RELAY_PIN, HIGH); // Pump OFF at power-on (active-low relay)
```

```
Serial.println("=====");

Serial.println("  Smart Irrigation System -- Ready");

Serial.println("=====");

Serial.println(" Soil DRY threshold  : " + String(SOIL_DRY_THRESHOLD) + " %");
Serial.println(" Soil WET threshold   : " + String(SOIL_WET_THRESHOLD) + " %");
Serial.println(" Tank LOW  level    : " + String(TANK_LOW_CM) + " cm");
Serial.println(" Tank FULL level   : " + String(TANK_FULL_CM) + " cm");
Serial.println(" Read interval     : " + String(READ_INTERVAL / 1000) + " s");
Serial.println(" Max pump duration : " + String(PUMP_MAX_DURATION / 1000) +
"s");

Serial.println("=====\\n");


connectWiFi();

Blynk.config(BLYNK_AUTH_TOKEN);

Blynk.connect();


mqttClient.setServer(MQTT_SERVER, MQTT_PORT);

connectMQTT();

}


// =====

// MAIN LOOP
```

```
// =====  
  
void loop() {  
  
    // Keep MQTT connection alive between publishes  
    if (!mqttClient.connected()) {  
        connectMQTT();  
    }  
    mqttClient.loop();  
    Blynk.run();  
  
    // 1. Read soil moisture  
    int soilRaw    = analogRead(SOIL_PIN);  
    int soilPercent = map(soilRaw, 4095, 0, 0, 100);  
    soilPercent    = constrain(soilPercent, 0, 100);  
  
    // 2. Read water level via ultrasonic  
    float distance = readUltrasonic();  
  
    // 3. Convert raw distance to a 0–100% tank fill value  
    int tankPercent = getTankPercent(distance);  
  
    // 4. Get human-readable tank status label  
    String tankStatus = getTankStatus(distance);
```

```
// 5. Run pump control logic (includes 30-second safety cut-off)

controlPump(soilPercent, distance);


// 6. Serial monitor output

printStatus(soilRaw, soilPercent, distance, tankPercent, tankStatus);


// 7. Publish to ThingSpeak via MQTT

publishAll(soilPercent, tankPercent, pumpRunning ? 1 : 0);


delay(READ_INTERVAL);
}


// =====

// WIFI CONNECTION

// =====

void connectWiFi() {

    Serial.print("Connecting to WiFi: ");

    Serial.println(WIFI_SSID);

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    int attempts = 0;

    while (WiFi.status() != WL_CONNECTED && attempts < 20) {
```

```
    delay(500);

    Serial.print(".");

    attempts++;

}

if (WiFi.status() == WL_CONNECTED) {

    Serial.println("\nWiFi connected.");

    Serial.println("IP address: " + WiFi.localIP().toString());

} else {

    Serial.println("\nWiFi connection failed. Running in offline mode.");

}

}

// =====

// MQTT CONNECTION (ThingSpeak)

// =====

void connectMQTT() {

    if (WiFi.status() != WL_CONNECTED) return;

    Serial.print("Connecting to ThingSpeak MQTT...");

    int attempts = 0;

    while (!mqttClient.connected() && attempts < 5) {
```

```
    if (mqttClient.connect(MQTT_CLIENT_ID, MQTT_USERNAME,
MQTT_PASSWORD)) {
        Serial.println(" connected.");
    } else {
        Serial.print(" failed (rc=");
        Serial.print(mqttClient.state());
        Serial.println("). Retrying in 2 s...");
        delay(2000);
        attempts++;
    }
}

// =====

// PUBLISH TO THINGSPEAK VIA MQTT

// Payload format: field1=<val>&field2=<val>&field3=<val>

// Field 1 = Soil Moisture (%)

// Field 2 = Tank Fill (%)

// Field 3 = Pump Status (1 = ON, 0 = OFF)

// =====

void publishToThingSpeak(int soilPercent, int tankPercent, int pumpStatus) {
    if (!mqttClient.connected()) {
        Serial.println(" [MQTT] Not connected — skipping publish.");
    }
}
```



```
    return;
}

String topic = "channels/" + String(CHANNEL_ID) + "/publish";

String payload = "field1=" + String(soilPercent)
    + "&field2=" + String(tankPercent)
    + "&field3=" + String(pumpStatus);

if (mqttClient.publish(topic.c_str(), payload.c_str())) {
    Serial.println(" [MQTT] Published: " + payload);
} else {
    Serial.println(" [MQTT] Publish failed.");
}
}

//Blynk

void publishAll(int soilPercent, int tankPercent, int pumpStatus) {

    // ThingSpeak via MQTT (existing — for data logging)
    publishToThingSpeak(soilPercent, tankPercent, pumpStatus);

    // Blynk (new — for live mobile dashboard)
    if (Blynk.connected()) {
```

```
Blynk.virtualWrite(V0, soilPercent);

Blynk.virtualWrite(V1, tankPercent);

Blynk.virtualWrite(V2, pumpStatus * 255); // LED: 0 = off, 255 = full brightness

if (pumpRunning) {

    unsigned long elapsed = (millis() - pumpStartTime) / 1000;

    Blynk.virtualWrite(V3, elapsed);

} else {

    Blynk.virtualWrite(V3, 0);

}

Serial.println(" [Blynk] Dashboard updated.");

} else {

    Serial.println(" [Blynk] Not connected — skipping.");

}

}

// =====

// READ ULTRASONIC

// Returns distance in cm, or -1 on timeout / out-of-range

// =====

float readUltrasonic() {

    digitalWrite(TRIG_PIN, LOW);

    delayMicroseconds(2);

    digitalWrite(TRIG_PIN, HIGH);
```

```
    delayMicroseconds(10);

    digitalWrite(TRIG_PIN, LOW);

    long duration = pulseIn(ECHO_PIN, HIGH, 30000); // 30 ms timeout

    if (duration == 0) return -1;

    float cm = (duration * 0.0343) / 2.0;

    if (cm < 2 || cm > 400) return -1;

    return cm;
}

// =====

// TANK FILL PERCENTAGE

// Converts raw distance (cm) to a 0–100% fill level.

// TANK_FULL_CM → 100%, TANK_LOW_CM → 0%

// Matches coursework Test 02 expected output:

// empty = 0%, half = 50%, full = 100%

// =====

int getTankPercent(float distance) {

    if (distance < 0)        return -1; // sensor error
```

```
    if (distance <= TANK_FULL_CM) return 100;

    if (distance >= TANK_LOW_CM) return 0;

    // Linear interpolation between FULL and LOW

    // Distance increases as water level falls, so invert the ratio.

    float ratio = (TANK_LOW_CM - distance) / (float)(TANK_LOW_CM -
TANK_FULL_CM);

    return (int)(ratio * 100);
}

// =====

// TANK STATUS LABEL

// =====

String getTankStatus(float distance) {

    if (distance < 0) return "UNKNOWN";

    if (distance <= TANK_FULL_CM) return "FULL";

    if (distance >= TANK_LOW_CM) return "LOW / EMPTY";

    return "MEDIUM";

}

// =====

// PUMP CONTROL LOGIC

//
```

```
// Turn ON when: soil is DRY (<40%) AND tank has water
// Turn OFF when: soil is WET (>=60%) OR tank is empty/error
//          OR 30-second maximum run duration reached
// =====

void controlPump(int soilPercent, float distance) {

    // Safety: stop pump if tank empty or sensor error
    if (distance < 0 || distance >= TANK_LOW_CM) {
        if (pumpRunning) {
            pumpOFF("Tank empty or sensor error");
        }
        return;
    }

    // Safety: enforce 30-second maximum pump duration (per Test 03)
    if (pumpRunning && (millis() - pumpStartTime >= PUMP_MAX_DURATION)) {
        pumpOFF("30 s duration limit reached");
        return;
    }

    // Stop pump once soil moisture reaches wet threshold
    if (pumpRunning && soilPercent >= SOIL_WET_THRESHOLD) {
        pumpOFF("Soil wet -- " + String(soilPercent) + "%");
    }
}
```

```
    return;
}

// Start pump when soil is dry and tank has sufficient water
if (!pumpRunning && soilPercent < SOIL_DRY_THRESHOLD) {
    pumpON("Soil dry -- " + String(soilPercent) + "%");
    return;
}
}

// =====
// PUMP ON / OFF
// =====

void pumpON(String reason) {
    digitalWrite(RELAY_PIN, LOW); // Active-low: LOW energises relay
    pumpRunning = true;
    pumpStartTime = millis();
    Serial.println(" >>> PUMP ON -- " + reason);
}

void pumpOFF(String reason) {
    digitalWrite(RELAY_PIN, HIGH); // Active-low: HIGH de-energises relay
    pumpRunning = false;
```

```
    Serial.println(" >>> PUMP OFF -- " + reason);
}

// =====

// SERIAL MONITOR OUTPUT

// =====

void printStatus(int soilRaw, int soilPercent, float distance,
                 int tankPercent, String tankStatus) {
    Serial.println("-----");
    Serial.print(" Soil RAW    : "); Serial.println(soilRaw);
    Serial.print(" Soil Moisture : "); Serial.print(soilPercent); Serial.println(" %");

    if (distance < 0) {
        Serial.println(" Water Level  : -- (timeout / out-of-range)");
        Serial.println(" Tank Fill    : --");
    } else {
        Serial.print(" Water Level  : "); Serial.print(distance, 1); Serial.println(" cm");
        Serial.print(" Tank Fill    : "); Serial.print(tankPercent); Serial.println(" %");
    }

    Serial.print(" Tank Status  : "); Serial.println(tankStatus);
    Serial.print(" Pump          : "); Serial.println(pumpRunning ? "ON <<<" : "OFF");
}
```

```
if (pumpRunning) {  
    unsigned long elapsed = (millis() - pumpStartTime) / 1000;  
    unsigned long remaining = (PUMP_MAX_DURATION / 1000) - elapsed;  
    Serial.print(" Pump Runtime  : "); Serial.print(elapsed);  Serial.println(" s");  
    Serial.print(" Auto-off in  : "); Serial.print(remaining); Serial.println(" s");  
}  
  
Serial.println("-----\n");  
}
```